SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA04
COURSE NAME: Operating Systems

COURSE OUTCOME COVERED

CO1: *Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments.*

QUESTIONS: 5 **Total Marks:100**

Annexure A

SHORT ANSWER TEST

85/100 A

Code of Conduct:

I ARJUN A. B/AI1126O006 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Arju

RUBRICS FOR CALCULATION

Criteria	Weightage (Marks)	Level 4 – Exemplary (Full Marks)	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	35 Marks	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25 Marks	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15 Marks	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15 Marks	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10 Marks	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

CSA04 OPERATING SYSTEMS

CO-1 AT-1

SHORT ANSWER TEST

QUESTION

1. Suppose a modern Operating System supports multiple users and processes with shared resources like CPU, memory, and I/O devices.

(a) Classify different types of Operating Systems suitable for this scenario and justify your choice.

(b) Draw the structure of an Operating System showing how resources are managed.

(c) Write a sequence of system calls required to create and execute a new process in such an environment.

ANSWER:

a) Types of Operating Systems and Justification

1. Multi-user OS: Allows multiple users to access the system simultaneously. Example: UNIX.
2. Multitasking OS: Allows multiple processes to execute concurrently by sharing CPU time. Example: Windows.
3. Multiprocessing OS: Uses multiple CPUs or cores to execute processes efficiently.
4. Time-sharing OS: Allocates CPU time to different users and processes for quick response.

Justification: These operating systems help manage shared CPU, memory, and I/O devices efficiently.

(b) Structure of an Operating System

Users and Application Programs ↓ System Call Interface ↓ Operating System Kernel

- Process Management
- Memory Management
- File Management
- I/O Management

↓ Hardware (CPU, Memory and I/O Devices)

Explanation: The kernel manages processes, allocates memory, schedules the CPU, and controls I/O devices.

(c) Sequence of System Calls to Create and Execute a Process The following

system calls are used in UNIX/Linux:

1. fork() – Creates a child process.
2. exec() – Loads and executes a new program.
3. wait() – Makes the parent process wait for the child process to finish.
4. exit() – Terminates the process.

Sequence: fork() → exec() → wait() → exit()

Conclusion: The Operating System manages multiple users and processes by sharing CPU time, allocating memory, and controlling I/O devices efficiently.

2. Consider an Operating System managing multiple files and devices for concurrent

users.

(a) Categorize system calls used for file and device management. (b)

Illustrate how the OS handles file operations using a layered structure

diagram.

(c) Write a pseudo-code or C-based system call sequence to open, read, and close a file.

ANSWER

(a) Categories of System Calls Used for File and Device Management

1. File Management: Used to create, open, read, write, and close files. Examples: open(), read(), write(), close().
2. Device Management: Used to access and control I/O devices. Examples: ioctl(), read(), write().
3. Information Maintenance: Used to obtain file information and attributes. Examples: stat(), fstat().
4. Protection Management: Controls access permissions for files and devices. Examples: chmod(), access().

(b) Layered Structure of File Operations

Users and Application Programs ↓ System Call Interface ↓ File System Interface ↓

Logical File System ↓ File Organization Module ↓ Basic File System ↓ Device Drivers ↓ Hardware (Disk and I/O Devices)

Explanation: Each layer performs a specific function. The file system manages file names, directories, permissions, and storage, while device drivers communicate with hardware.

(c) C Program to Open, Read, and Close a File

```
#include <stdio.h>

#include <fcntl.h>

#include <unistd.h> int

main()

{    int

fd;

   char buffer[100];  fd =

open("sample.txt", O_RDONLY);  if (fd

< 0)

   { printf("File opening failed\n");

return 1;

   }

read(fd, buffer, sizeof(buffer) - 1);

close(fd);    printf("File read

successfully\n");  return 0;

}
```

Explanation:

1. open() opens the file in read-only mode.
2. read() reads data from the file.
3. close() closes the file after reading.

Conclusion: File and device management system calls allow the Operating System to access files, control devices, and share resources among concurrent users.

3. Assume a real-time Operating System used in an embedded system controlling industrial machinery.

(a) Identify the type of OS and explain why it is suitable for real-time applications.

(b) Draw a diagram representing the interaction between hardware, kernel, and user processes.

(c) List and explain system calls required for process synchronization in this system.

ANSWER:

The suitable operating system is a Real-Time Operating System (RTOS).

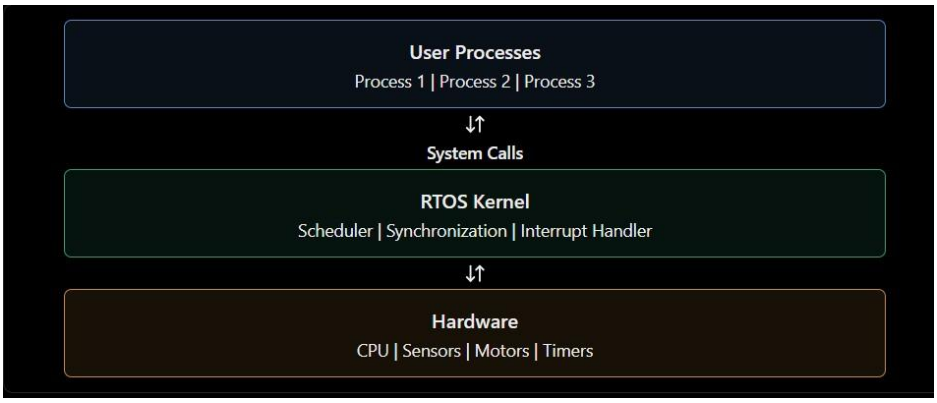
An RTOS is an operating system designed to execute tasks and respond to events within a specified time limit.

Reasons for its suitability:

1. Deterministic response: Provides predictable response times for critical operations.
2. Priority-based scheduling: Executes high-priority tasks before less important tasks.
3. Low latency: Responds quickly to sensor signals and hardware interrupts.
4. Reliability: Supports continuous and stable operation of industrial machinery.
5. Deadline management: Ensures critical tasks are completed within their specified deadlines.

Example: Industrial robots, automated production lines and machine-control systems.

(b) Draw a diagram representing the interaction between hardware, kernel and user processes.



Explanation:

- User processes: Execute application tasks such as monitoring temperature and controlling motors.
- Kernel: Manages CPU scheduling, synchronization, interrupts and hardware resources.
- Hardware: Includes the processor, sensors, timers and actuators that interact with the physical environment.
- System calls: Provide an interface through which user processes request services from the kernel.

(c) List and explain system calls required for process synchronization.

Process synchronization coordinates multiple processes accessing shared resources to prevent conflicts and maintain correct execution.

1. Semaphore operations (wait() and signal()): Control access to shared resources. wait() acquires or waits for a semaphore, while signal() releases it or notifies a waiting process.

2. Mutex operations (lock() and unlock()): Ensure that only one process or thread enters a critical section at a time.

3. Condition variable operations (wait() and signal()): Allow a process to wait until a specific condition is satisfied and notify waiting processes when the condition changes.

4. Message-passing operations (send() and receive()): Allow processes to exchange information and coordinate their activities.

5. Process suspension and resumption: Operations such as sleep() and wake-up suspend a process and resume it when the required event occurs.

Conclusion: An RTOS uses synchronization mechanisms to prevent race conditions, coordinate processes and ensure the timely and reliable operation of industrial machinery.

4. A distributed computing system consists of multiple interconnected computers sharing resources over a network.

(a) Classify the type of Operating System used and compare it with centralized OS.

(b) Draw the architecture of a distributed OS showing communication between nodes.

(c) Write the sequence of system calls involved in inter-process communication (IPC).

ANSWER:

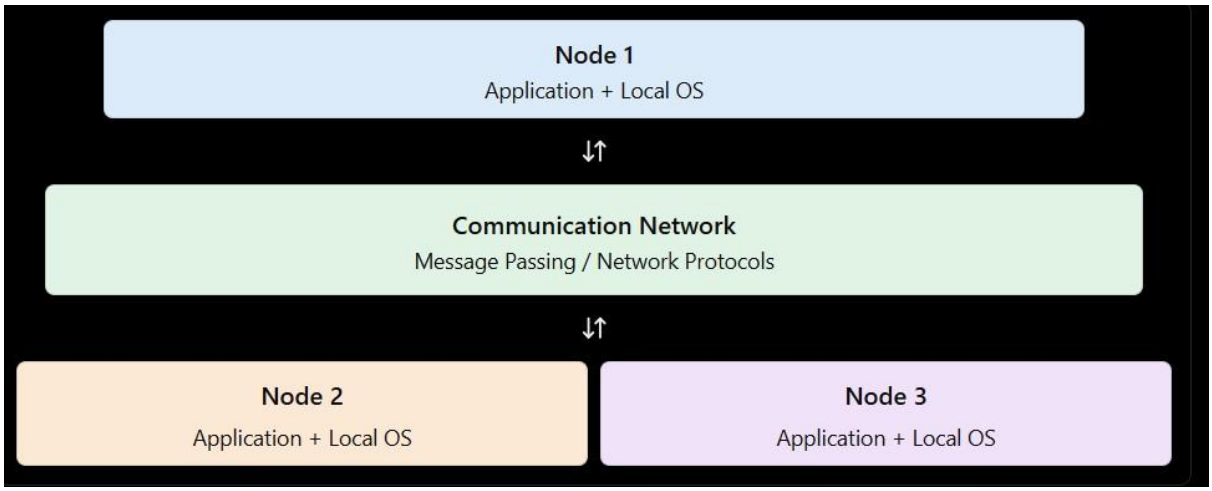
The operating system used is a Distributed Operating System (Distributed OS).

A Distributed OS manages multiple interconnected computers and makes them work together as a single system by sharing resources and communicating through a network.

Comparison between Distributed OS and Centralized OS

Distributed OS	Centralized OS
Runs across multiple interconnected	Runs on a central computer. computers.
Resources are distributed among multiple nodes.	Resources are managed by a central system.
Provides better resource sharing across	Resources are mainly managed centrally. nodes.
Failure of one node may not stop the entire system.	Failure of the central system may affect the whole system.
Can be expanded by adding more nodes.	Expansion may require upgrading the central system.

(b) Draw the architecture of a Distributed OS showing communication between nodes.



Explanation:

- Nodes: Independent computers that perform processing tasks.
- Local OS: Manages the resources and processes of each node.
- Communication network: Connects the nodes and enables data exchange.

- Message passing: Allows processes on different nodes to communicate and share resources.

(c) Write the sequence of system calls involved in IPC.

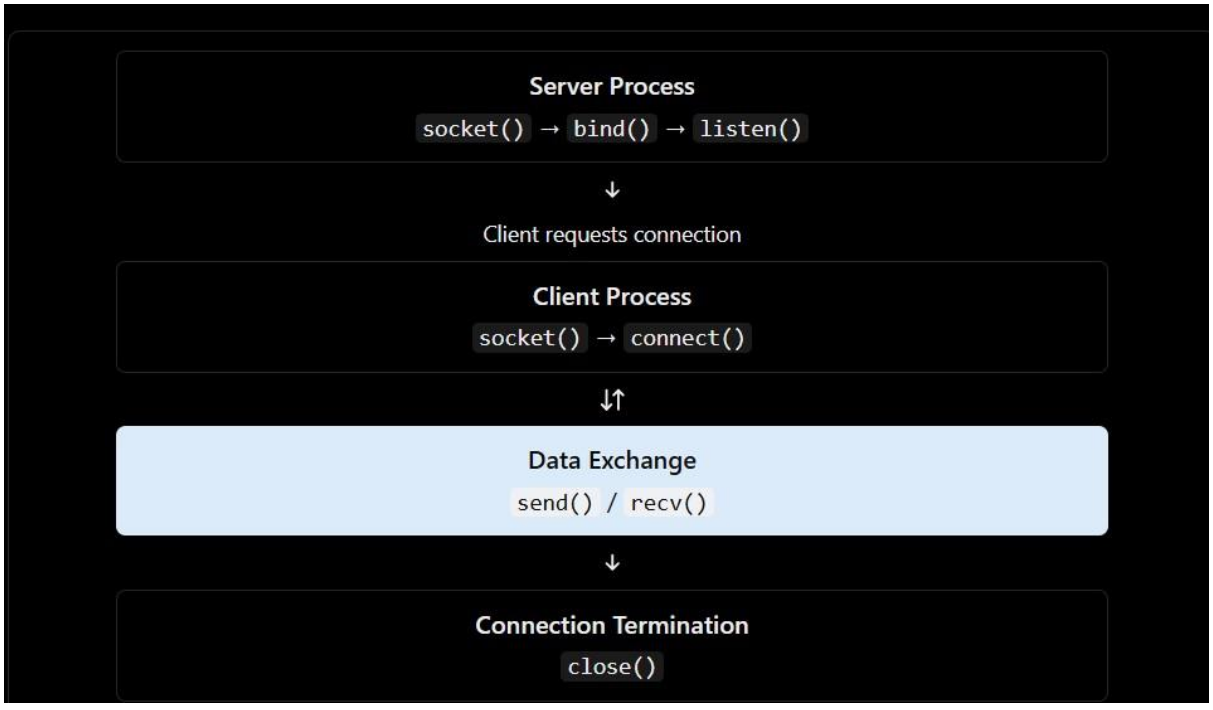
Answer:

Inter-Process Communication (IPC) allows processes to exchange data and synchronize their execution. In a distributed system, IPC can be implemented using sockets.

The sequence of system calls for communication between two processes is:

1. socket() – Creates a communication endpoint.
2. bind() – Assigns an address to the server socket.
3. listen() – Makes the server ready to accept connections.
4. accept() – Accepts a connection request from a client.
5. connect() – Allows the client to connect to the server.
6. send() / write() – Sends data between processes.
7. recv() / read() – Receives data from another process.
8. close() – Closes the communication connection.

Sequence diagram to draw:



Conclusion: A Distributed OS enables resource sharing and communication among multiple computers, while IPC system calls allow processes to exchange information efficiently.

5. Consider a scenario where an Operating System must manage process scheduling, memory allocation, and device communication efficiently.

(a) Explain the major functions of an OS in handling these tasks.

(b) Draw a block diagram representing OS components and their interactions.

(c) Write system call examples for memory allocation and device management.

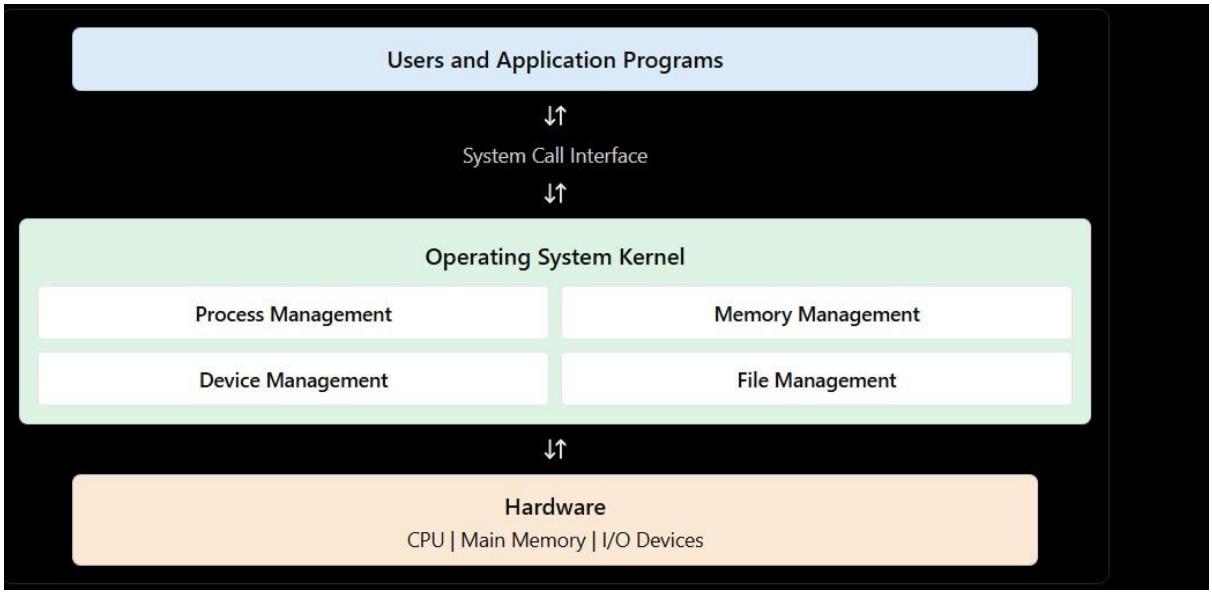
ANSWER:

(a) An Operating System (OS) is system software that acts as an interface between the user and computer hardware. It manages system resources and ensures efficient execution of programs.

The major functions are:

1. Process Management: Creates, schedules and terminates processes. It allocates CPU time using scheduling algorithms such as FCFS, SJF and Round Robin.
2. Memory Management: Allocates and deallocates memory to processes. It tracks memory usage and ensures proper utilization of available memory.
3. Device Management: Controls input and output devices through device drivers. It manages devices such as keyboards, printers and disks.
4. File Management: Creates, stores, retrieves and organizes files and directories.
5. Security and Protection: Prevents unauthorized access and protects system resources.

(b)



Explanation:

- Application programs request services from the OS.
- System call interface provides access to OS services.
- Kernel manages processes, memory, devices and files.
- Hardware performs processing, stores data and handles input/output operations.

(c) System calls allow application programs to request services from the operating system.

1. Memory allocation system calls

- mmap() – Maps memory into a process's address space.
- munmap() – Unmaps previously mapped memory.
- brk() / sbrk() – Changes the size of a process's data segment on systems that support them.

Example in C (POSIX/Linux):

```
#include <sys/mman.h>
#include <stdio.h> int main() {
void *ptr = mmap(NULL, 4096,
                 PROT_READ | PROT_WRITE,
                 MAP_PRIVATE | MAP_ANONYMOUS,
                 -1, 0);
  if (ptr == MAP_FAILED)
      return 1;
munmap(ptr, 4096);
    return 0;
}
```

Here, mmap() allocates a memory mapping of 4096 bytes, and munmap() releases it.

2. Device management system calls

- open() – Opens a device file.
- read() – Reads data from a device.
- write() – Sends data to a device.

- ioctl() – Performs device-specific control operations.
- close() – Closes the device file.

Example in C (POSIX/Linux):

```
#include <fcntl.h>
#include <unistd.h> int main() {        int fd =
open("/dev/example", O_WRONLY);

    if (fd == -1)
        return 1;
  write(fd, "A", 1);
    close(fd);
    return 0;
    }
```

Here, open() opens the device, write() sends data, and close() closes the device. /dev/example is a placeholder for an actual device file.